

TP 4.b – Setup n8n (stack mutualisée)

Pré-requis commun aux deux runbooks suivants :

- `tp4b-n8n-research-assistant.md` – Notebook 1 reconstruit en no-code
- `tp4b-n8n-morning-briefing.md` – Notebook 2 reconstruit en no-code

Durée : ~ 15 min · **Coût** : 0 € · **Format** : runbook formateur, à exécuter une seule fois.

Sommaire

- [0. Pré-requis](#)
- [Étape 1 – Lancer la stack Docker \(n8n + Postgres + Qdrant\)](#)
- [Étape 2 – Premier accès à n8n + création du compte owner](#)
- [Pièges classiques rencontrés \(notes formateur\)](#)
- [Et après ?](#)

0. Pré-requis

Élément	Vérification
Docker Desktop installé et démarré	<code>docker info --format '{{.ServerVersion}}'</code> renvoie un numéro
~ 4 Go RAM libres	Activity Monitor / Task Manager
Ports libres : <code>5678</code> (n8n), <code>6333</code> (Qdrant), <code>5432</code> (Postgres)	<code>lsof -nP -iTCP:5678 -sTCP:LISTEN</code> (rien = libre)
Terminal positionné dans <code>04 Agents IA/TP4-b/</code>	<code>cd "04 Agents IA/TP4-b"</code>

Trame formateur : « Docker Desktop n'est pas obligatoire pour n8n – on pourrait l'installer avec npm. Mais Docker apporte la portabilité (`docker-compose.yml` partageable) et l'isolation (Postgres + Qdrant inclus). C'est le setup qu'on retrouve en prod. »

Étape 1 – Lancer la stack Docker (n8n + Postgres + Qdrant)

1.1 – Créer `docker-compose.yml`

Fichier `04 Agents IA/TP4-b/docker-compose.yml` :

```
version: "3.8"
```

```
services:
  postgres:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_USER: n8n
      POSTGRES_PASSWORD: n8npass
      POSTGRES_DB: n8n
    volumes:
      - postgres_data:/var/lib/postgresql/data

  qdrant:
    image: qdrant/qdrant:latest
    restart: unless-stopped
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_data:/qdrant/storage

  n8n:
    image: n8nio/n8n:latest
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: postgres
      DB_POSTGRESDB_DATABASE: n8n
      DB_POSTGRESDB_USER: n8n
      DB_POSTGRESDB_PASSWORD: n8npass
      N8N_HOST: localhost
      N8N_PORT: 5678
      N8N_PROTOCOL: http
      WEBHOOK_URL: http://localhost:5678/
      GENERIC_TIMEZONE: Europe/Paris
      N8N_DIAGNOSTICS_ENABLED: "false"
    depends_on:
      - postgres
      - qdrant
    volumes:
      - n8n_data:/home/node/.n8n
```

```
volumes:
  postgres_data:
  qdrant_data:
  n8n_data:
```

1.2 – Valider le YAML avant de lancer

```
docker compose config && echo "YAML OK"
```

Doit afficher la config résolue puis `YAML OK`. Un warning `the attribute "version" is obsolete` est **inoffensif** (Compose v2 ignore cet attribut).

1.3 – Démarrer la stack

```
docker compose up -d
```

Premier lancement → ~ 1-3 min de `docker pull` pour les 3 images.

1.4 – Vérifier

```
docker compose ps
curl -s http://localhost:6333/collections
curl -sI http://localhost:5678 | head -1
```

Attendu :

- 3 services à l'état `Up` : `tp4-b-n8n-1`, `tp4-b-postgres-1`, `tp4-b-qdrant-1`
- Qdrant → `{"result":{"collections":[]},"status":"ok",...}`
- n8n → `HTTP/1.1 200 OK`

Trame pédagogique (Étape 1)

« Trois services, une commande. Ce même `docker-compose.yml` tourne en local pour le TP, sur un VPS à 5 €/mois en prod, ou sur n8n Cloud. La portabilité, c'est l'argument #1 contre Power Automate (qui n'a pas d'équivalent off-cloud). »

Points à souligner :

- **Postgres** = base interne de n8n (workflows, exécutions, **credentials chiffrés AES**) – n8n s'occupe du chiffrement, on n'a rien à faire.
- **Qdrant** = base vectorielle pour le RAG (utile pour A ; inactif mais inoffensif pour B).
- **n8n** = orchestrateur + UI sur `localhost:5678`.
- Le port `5432` Postgres **n'est pas exposé** : seul n8n y accède via le réseau Docker interne. Bon réflexe sécurité.
- `N8N_DIAGNOSTICS_ENABLED=false` désactive la télémétrie – important si on traitera de vraies données.

Question typique du public à anticiper :

« Si je redémarre Docker, je perds tout ? »

→ **Non**. Les volumes nommés (`postgres_data`, `qdrant_data`, `n8n_data`) persistent. `docker compose down` arrête les conteneurs mais **garde** les volumes. Seul `docker compose down -v` détruit les volumes (commande dangereuse à mentionner).

Étape 2 – Premier accès à n8n + création du compte owner

2.1 – Ouvrir l'UI

Dans le navigateur : <http://localhost:5678>

Premier démarrage → écran « **Set up owner account** ».

2.2 – Créer le compte propriétaire

Champ	Contenu
Email	libre, login local uniquement (aucun envoi externe)
First / Last name	libre
Password	min. 8 caractères, 1 majuscule, 1 chiffre

→ **Next**.

2.3 – Skip des écrans optionnels

- Écran « Personnalisation » (type d'usage, équipe...) → **Skip** (lien discret en bas).
- Écran « Get a free license key » → **Skip** également. La license débloquent uniquement des features Enterprise (folders, RBAC) inutiles pour le TP.

2.4 – Vérifier l'accueil

Tu arrives sur le canvas **Workflows** (vide). Le menu de gauche doit montrer :

- **Workflows**
- **Credentials**
- **Executions**
- **Templates**

✅ Si ces 4 menus sont là → Étape 2 OK.

Trame pédagogique (Étape 2)

« Le compte owner est purement local – il vit dans le Postgres qu'on a démarré il y a 2 minutes. Aucune télémétrie, aucun cloud. C'est un argument fort vs Power Automate, qui exige un tenant Microsoft. Cette instance pourrait tourner air-gapped sur un serveur sans Internet (sauf pour appeler le LLM). »

« Le panneau "Executions" est l'équivalent du `verbose=True` de LangChain ou du `print()` de debug – sauf qu'il est natif, persistant, et visuel. Chaque node garde ses inputs/outputs en JSON pendant 14 jours par défaut. »

Pièges classiques rencontrés (notes formateur)

Piège #1 – Erreur YAML « did not find expected key »

Symptôme :

```
yaml: line 2: did not find expected key
```

Causes fréquentes (par ordre de probabilité) :

1. **Tabulation** au lieu d'espaces – YAML interdit strictement les tabulations.
2. **Mauvaise indentation** d'une clé top-level (par exemple `services:` indenté de 2 espaces alors qu'il devrait être à la colonne 0).
3. **Deux-points manquant** sur une clé.
4. **Caractères invisibles** collés depuis un PDF / Word / une page Notion.

Diagnostic :

```
# Voir tabs et fins de ligne
cat -A docker-compose.yml | head -10
# ^I = tabulation
# $ = fin de ligne (espaces avant = trailing whitespace)

# Valider la syntaxe sans démarrer
docker compose config
```

Solution la plus fiable : recréer le fichier avec un heredoc (insensible aux substitutions shell et aux auto-corrrections d'éditeur) :

```
cat > docker-compose.yml <<'EOF'
version: "3.8"
...
EOF
```

À dire à l'audience : « C'est LE piège n°1 sur Docker Compose. Anticipez-le : `docker compose config` valide la syntaxe avant tout démarrage. Si vous éditez du YAML en démo

live, utilisez `cat <<EOF` plutôt qu'un éditeur GUI – pas de tabulation parasite. »

Piège #2 – `docker compose up` ne trouve pas le fichier

Symptôme : `no configuration file provided`.

Cause : la commande tourne dans un dossier sans `docker-compose.yml`.

Solution : `cd` dans le bon dossier (où vit le fichier) **ou** spécifier le chemin avec `-f` :

```
docker compose -f /chemin/absolu/docker-compose.yml up -d
```

Piège #3 – Port déjà occupé

Symptôme : `bind: address already in use` (port `5678`, `6333` ou `5432`).

Diagnostic :

```
lsof -nP -iTCP:5678 -sTCP:LISTEN
```

Solution : tuer le processus, ou changer le mapping côté hôte dans `docker-compose.yml` :

ports:

```
- "5679:5678" # n8n accessible sur localhost:5679 à l'extérieur
```

Piège #4 – Docker Desktop éteint

Symptôme : `Cannot connect to the Docker daemon`.

Solution : ouvrir l'app **Docker Desktop**, attendre que la baleine soit stable dans la barre de menu.

Et après ?

La stack est partagée entre les deux runbooks. Choisis ton parcours :

Variante	Fichier	Ce qu'on construit
A – Research Assistant	tp4b-n8n-research-assistant.md	Agent IA avec 3 outils (RAG Qdrant + Tavily + arXiv), Webhook → Agent → Réponse JSON
B – Morning Briefing	tp4b-n8n-morning-briefing.md	Briefing matinal Gmail + Calendar + Drive + Slack, Schedule Trigger 8h30

Recommandation formateur : commencer par **A** (plus court, pas d'OAuth Google), puis **B** pour démontrer la valeur des connecteurs natifs M365/Google.

Annexe – Commandes Docker utiles

```
# Voir les logs d'un service en direct
docker compose logs -f n8n

# Redémarrer un service sans tout casser
docker compose restart n8n

# Stopper la stack (les volumes restent)
docker compose down

# RESET COMPLET (efface tous les workflows, credentials, vecteurs) –
DANGEREUX
docker compose down -v

# Voir l'usage RAM/CPU des conteneurs
docker stats
```

Livrables à ce stade

-  `docker-compose.yml` versionné dans `04 Agents IA/TP4-b/`
-  Stack `n8n + postgres + qdrant` à l'état `Up`
-  Compte owner n8n créé sur `http://localhost:5678`
-  UI n8n accessible avec le menu Workflows / Credentials / Executions / Templates